

6주차 밤에 한 번 더 세는 이유 — 배치와 확정 통계

6주차 · 배치 처리

이 장의 수치는 모두 `practice/chapter6/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

한 가지 예외가 있다. Airflow 엔진이 직접 남긴 태스크 상태와 산출물 해시는 이 강의 환경에서 다시 돌릴 수 없어 **과거 실행이 남긴 산출물에서 옮긴 값**이며, 그 블록의 출처는

`practice/chapter6/data/output/` 으로 적었다.

□ 5주차에서 스트리밍(streaming)은 지금 벌어지는 일을 빠르게 썼지만, 워터마크(watermark)를 넘겨 도착한 이벤트를 폐기했음 ○ 그 폐기는 소리를 내지 않았고, 상황판에 남은 수치는 근사치였음 □ 이 장은 그다음 날 새벽에 같은 데이터를 다시 세는 일을 다룸 ○ 배치(batch)는 느린 대신 마감까지 들어온 것을 전부 담고, 한 번 낸 수치를 바꾸지 않음 ○ 그래서 기관이 제출하는 확정 통계는 배치가 만듦 □ 이 장도 설명과 실습을 나누지 않음 ○ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐 ○ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

□ 5주차에서 이미 실시간으로 세고 있는데, 밤에 왜 한 번 더 세는가? □ 원천 60건이 보고서에서 54건이 되었다면, 나머지 6건은 어디로 갔다고 답할 수 있는가? □ 배치가 중간에 실패했는데 보고서 파일이 만들어져 있다면, 무엇이 잘못된 것인가? □ 같은 날짜를 다시 계산했더니 수치가 달라졌다면, 어느 수치가 진짜인가?

2. 이 장에서 실행할 것

□ 배치는 가게가 하루를 마감하는 절차와 항목이 하나씩 대응함

장부 마감	이 장의 배치
영업 중 계산대 화면에 뜨는 잠정 매출	스트리밍 상황판의 근사치(5주차)
문을 닫은 뒤 전표를 전부 다시 세어 만든 일계표	마감 후 전량을 재계산한 확정 통계(3절)
전표를 한 장도 버리지 않고 정상·기재누락·코드없음·중복으로 나눠 둬	정제의 네 갈래 출구(4절)
세어 본 합이 전표 뭉치와 안 맞으면 마감하지 않음	총계 보존식이 깨지면 배치를 실패시킴(5절)
일계표를 모아 월계표를 만들 때 전표를 다시 세지 않음	주간 롤업(rollup)은 일별 확정 산출물만 읽음(6절)

표 6.1 장부 마감과 배치의 대응

- ☐ 비유는 여기까지다 ○ 장부는 사람이 마감하므로 합이 안 맞으면 조정 전표를 끼워 넣어 억지로 맞출 수 있음 ○ 사전에 없는 지역 표기를 "가장 비슷한 자치구"로 바꿔 합을 맞추는 것은 데이터 조작이며, 이 장의 설계는 맞추는 대신 멈춤 ○ 그 멈추는 장면이 5절에서 숫자로 나타남
- ☐ 실행하기 전에 네 가지를 먼저 적음. 남이 만든 산출물을 먼저 보면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 6.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
사흘치 원천 파일의 물리 레코드 합			
지역명을 그대로 키로 집계하면 몇 줄이 되는가			
입력 파일이 없는 날, 다섯 태스크 중 실제로 실행되는 것은 몇 개인가			
주간 집계만 정제 규칙이 구버전이면 매핑 건수가 몇 건 어긋나는가			

- ☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	6-2-cutoff-scope.py	구간 경계 확인과 전수 중복 검사	3	없음
2	6-3-clean-accounting.py	표준 코드 매핑과 총계 보존식	4	없음
3	6-4-dag-order.py	태스크 순서·실패 전파·품질 게이트	5	없음
4	6-5-rerun-rollup.py	재실행 해시 비교와 주간 롤업	6	data/output/daily/ (저장소에 포함)
선택	6-1-daily-report-dag.py	같은 절차를 실제 Airflow 엔진으로	5-6	macOS·Linux·WSL2

시작 전 준비

□ 1~4번은 표준 라이브러리만 쓰므로 설치할 패키지가 없음(실습 검증 환경은 Python 3.13) □ 선택 실습만 Airflow 3.3.0이 필요하고, Airflow는 Windows 네이티브를 지원하지 않음 ○ 아래 한 줄이 운영체제·네트워크·패키지를 점검하고 가상환경을 준비함

```
python scripts/setup_practice.py 6
```

3. 배치의 마감은 달력이 정한다

□ 같은 민원을 세는데 왜 두 시스템이 필요한지는 산출물의 성격에서 갈림 ○ 상황판의 수치는 빨라야 하지만 나중에 바뀌어도 됨 ○ 기관이 제출한 통계는 늦어도 되지만 제출한 뒤에 바뀌면 안 됨 ○ 한 시스템으로 두 요구를 함께 만족시킬 수 없어 스트리밍과 배치를 나란히 둠

□ 두 방식의 핵심 차이는 **마감을 정하는 주체**임 ○ 스트리밍에서는 데이터가 마감을 정함 – 5주치의 워터마크는 관측된 가장 늦은 이벤트 시각에서 허용 지연만큼을 뺀 시점을 마감으로 삼았음 – 그 선을 넘겨 도착한 21:49 이벤트는 폐기되어 집계에 영영 들어가지 못했음

(practice/chapter5/data/output/ch5_late_event_report.json 의 dropped_by_watermark_cumulative) ○

배치에서는 달력이 마감을 정함 – "7월 1일분은 7월 2일 새벽에 마감한다"고 정해 두면, 그때까지 도착한 지연분은 늦었든 아니든 전부 그 구간에 들어감

□ 여기서 배치의 세 성질이 따라 나옴 ○ **확정성** — 구간 전체를 한 번에 계산하므로 산출물이 그 입력에 대한 최종 결과가 됨 – 스트리밍이 실시간으로 낸 근사치를 사후에 검증하는 기준선 역할을 함 ○ **전체 조망** — 그날 데이터를 통째로 쥐고 있으므로 전수 중복 검사와 원천 대사가 성립함 – 스트리밍은 흐르는 데이터의 창(window)만 보므로 창을 벗어난 쌍을 만나지 못함 ○ **재실행 가능성** — 실패하면 그 날짜를

통째로 다시 돌림 – 다시 돌린 결과가 기존 산출물 위에 이중으로 쌓이면 안 되므로, 4주차의 멍등성 (idempotency)이 배치의 전제로 되돌아옴 ○ 셋의 대가는 지연임. 일 배치라면 산출물이 최대 하루 늦고, 그 사이의 상황 판단은 여전히 스트리밍이 맞음

□ 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter6
python3 code/6-2-cutoff-scope.py
```

```
# 출처: practice/chapter6/results/6-2-cutoff-scope.log
```

[1] 입력 – 날짜 하나가 파일 하나다

```
2026-07-01.jsonl  물리 레코드 20건  첫 접수 08:00  마지막 접수 18:08
2026-07-02.jsonl  물리 레코드 18건  첫 접수 08:00  마지막 접수 17:04
2026-07-03.jsonl  물리 레코드 22건  첫 접수 08:00  마지막 접수 19:12
사흘 합계 60건
```

[2] 달력 마감 – 파일 안의 레코드가 모두 그 날짜인가

```
2026-07-01: 전부 같은 날짜
2026-07-02: 전부 같은 날짜
2026-07-03: 전부 같은 날짜
→ 구간의 경계를 데이터가 아니라 달력이 정함
```

[3] 전체 조망 – 하루치를 통째로 쥐면 전수 중복 검사가 성립한다

```
2026-07-02: C260702-001 가 두 번 – 08:00 과 11:12, 간격 192분
5주차 창 길이 10분 기준으로 같은 창에서 만나는가: 아니오
```

[4] 하루 안에서의 접수 분포 – 배치가 한 번에 보는 범위

```
2026-07-01: 접수 시각대 8시~18시, 시각대 11개에 걸침
2026-07-03: 접수 시각대 8시~19시, 시각대 12개에 걸침
```

[5] 지역 표기의 원문 – 정제 전에는 이렇게 들어와 있다

```
'강남구'      18건
'강남'        4건
None          2건
' 강남구 '    1건
'강남 '       1건
```

원문 표기 13가지 – 정제는 6-3에서 이어감

□ 화면에 나온 값에서 읽을 것 ○ **구간의 경계가 달력임** — 세 파일 어디에도 다른 날짜의 레코드가 섞여 있지 않음. "7월 1일분"이 무엇인지를 데이터의 도착 상황이 아니라 파일 이름이 정함 ○ **중복 두 건의 간격이 192분임** — 같은 민원 ID `c260702-001` 이 08:00과 11:12에 두 번 들어왔음. 5주차가 쓴 10분 창으로는 이 둘이 같은 창에서 만나지 못하므로 창 단위 처리로는 지울 수 없는 중복임 ○ **하루가 창 여럿에 흩어져 있음** — 7월 3일 접수는 8시부터 19시까지 시각대 12개에 걸쳐 있음. 배치는 그 전부를 한 번에 쥐므로 간격과 무관하게 전수 검사가 성립함 ○ **원문 표기가 13가지임** — 정제 전 지역 표기는 자치구 세 곳을 가리키는 데 13가지 형태로 들어와 있음. 4절이 여기서 출발함

- 두 수치가 다른 것 자체는 결함이 아니다
- 스트리밍의 근사치와 배치의 확정치는 마감 규칙이 다르므로 다를 수 있다.
- 결함인지 설계인지를 가르는 것은 **왜 다른지가 문서로 설명되는가**다. 설명되면 설계고, 설명되지 않으면 결함이다.

4. 표기가 흔들려도 총계는 맞아야 한다

□ 정제에서 가장 위험한 코드는 처리하지 못한 레코드를 조용히 건너뛰는 코드임 ○ 4주차의 유실도, 5주차의 워터마크 폐기도 스스로를 알리지 않았음 ○ 정제 단계의 누락도 마찬가지여서, 보고서 총계가 원천보다 조금 작는데 아무도 모르는 상태가 됨

□ 문제가 시작되는 자리는 지역명 표기임 ○ 같은 강남구 민원이 "강남구", " 강남구 " (앞뒤 공백), "강남" (약칭), "강남 " (뒤 공백)으로 적혀 있음 ○ 사람 눈에는 같은 곳이지만 이름을 키로 집계하면 한 자치구가 여러 줄로 쪼개짐 ○ 그래서 이름 대신 행정표준코드(법정동코드)를 키로 씀 - 코드를 쓰면 인구·예산 같은 다른 기관 데이터와도 같은 키로 붙일 수 있음

□ 정제는 모든 입력 레코드를 네 갈래 중 하나로 보내고 어디로 갔는지 기록함 ○ **매핑 성공(mapped)** — 표준 코드가 붙어 집계에 들어감 ○ **지역 미기재(missing)** — 지역 필드가 비어 있음. 건수와 민원 ID를 따로 기록함 ○ **매핑 실패(unmapped)** — 사전에 없는 표기임. 원문 그대로 따로 기록함 ○ **중복 제거(dup_removed)** — 같은 민원 ID의 재전송은 먼저 온 것만 남김(4주차의 원칙)

□ 네 갈래의 합이 입력과 맞는지 검사하는 식이 **총계 보존식**임

```
physical = mapped + missing + unmapped + dup_removed
valid    = physical - dup_removed = mapped + missing + unmapped
```

□ 두 번째 코드를 실행함. 6-1의 Airflow DAG와 같은 정제 규칙을 Airflow 없이 적용함

```
python3 code/6-3-clean-accounting.py
```

출처: practice/chapter6/results/6-3-clean-accounting.log

[2] 지역명을 그대로 키로 쓰면 - 사흘치 원문 표기 기준

집계 줄 수 13줄

강남 계열만 4줄로 쪼개짐: ' 강남구 ' 1건, '강남' 4건, '강남 ' 1건, '강남구' 18건

강남 계열 합 24건 - 어느 줄도 단독으로는 이 수를 보이지 못함

[3] 정규화 후 표준 코드를 키로 쓰면

집계 줄 수 3줄

11440 마포구: 16건

11620 관악구: 15건

11680 강남구: 23건

강남구 23건 - [2]의 강남 계열 합보다 1건 적음(중복 점수 제거분)

[4] 일별 정제 회계 - physical = mapped + missing + unmapped + dup_removed

날짜	physical	dup	valid	mapped	missing	unmapped	보존
----	----------	-----	-------	--------	---------	----------	----

2026-07-01	20	0	20	18	1	1	OK
------------	----	---	----	----	---	---	----

검산 $20 = 18 + 1 + 1 + 0$ 공백 보정 1건, 약칭 보정 4건

2026-07-02	18	1	17	16	0	1	OK
------------	----	---	----	----	---	---	----

검산 $18 = 16 + 0 + 1 + 1$ 공백 보정 1건, 약칭 보정 3건

2026-07-03	22	0	22	20	1	1	OK
------------	----	---	----	----	---	---	----

검산 $22 = 20 + 1 + 1 + 0$ 공백 보정 1건, 약칭 보정 3건

[5] 사흘 합계 - $60 = 54 + 2 + 3 + 1$ 보존 OK

미기재 C260701-019, C260703-021

미매핑 '판교', '성남시', '김포'

중복제거 C260702-001

[6] 조용히 건너뛰는 코드와 대조 - 매핑 실패를 기록하지 않으면

보고서에 적히는 총계 54건

원천 물리 레코드 60건

설명되지 않는 차이 6건

예외 없음, 종료 코드 0 - 차이가 로그에도 산출물에도 남지 않음

□ 화면에 나온 값에서 읽을 것 ○ 키를 바꾸면 13줄이 3줄이 됨 — 이름을 키로 쓰면 강남 계열만 4줄로 쪼개지고, 어느 줄도 단독으로는 강남구 전체를 보이지 못함. 표준 코드를 키로 쓰면 세 줄로 모임 ○ 강남 계열 합 24건과 매핑된 강남구 23건이 1건 다름 — 사라진 1건은 7월 2일의 재전송 C260702-001 임. 두 수가 다른 이유가 산출물 안에 적혀 있으므로 확인됨 ○ 사흘 전체가 $60 = 54 + 2 + 3 + 1$ 로 남김없이 설명됨

— 원천 60건 중 어느 것도 행방이 빈칸으로 남지 않음 ○ 조용히 버리면 차이 6건이 어디에도 남지 않음 —
매핑 실패를 세지 않는 코드는 보고서에 54건만 적고 종료 코드 0으로 끝남. 감사에서 가장 곤란한 결함이
이 형태임

□ 미기재와 미매핑을 합쳐 "실패 2건"으로 보고하지 않음 ○ 미기재는 접수 화면이 지역을 받지 못했다는
뜻이므로 조치 대상이 접수 시스템임 ○ 미매핑은 값은 들어왔는데 우리 사전에 없다는 뜻이므로 조치
대상이 사전임 ○ 실제로 걸린 판교·성남시·김포는 오타가 아니라 서울 자치구 사전 밖의 유효한 지명일
수 있음 – 관할이 넓어졌는지 접수 오류인지는 사람이 판단하며, 두 항을 합치면 그 판단에 필요한 정보가
사라짐

5. DAG와 품질 게이트 — 틀린 보고서를 아예 만들지 않는다

□ 정제·집계·보고서 생성을 한 스크립트에 이어 붙이면 운영에서 곧 막힘 ○ 어제 새벽 배치가 집계 도중에
죽었는데 어디서부터 다시 돌려야 하는지 알 수 없음 ○ 정제가 실패했는데 보고서 단계가 어제 파일을
읽고 그냥 돌아 버리지는 않았는지 알 수 없음 ○ 워크플로 오케스트레이터(orchestrator)는 이 문제를
절차의 구조를 바꿔 풀

□ 오케스트레이터는 절차를 **태스크(task)** 와 **의존성**의 그래프로 선언하게 함 ○ 이 그래프를 DAG(Directed
Acyclic Graph, 방향 비순환 그래프)라 함 ○ 방향은 "정제가 끝나야 집계한다"는 순서를 뜻하고, 비순환은
서로를 기다리는 교착이 없다는 보장임 ○ 순서 보장·실패 전파·재시도·스케줄은 엔진이 맡음

```
# practice/chapter6/code/6-1-daily-report-dag.py 의 의존성 선언
t_validate >> t_clean >> [t_aggregate, t_quality] >> t_report
```

□ 태스크 경계를 어디에 둘지는 두 가지가 정함 ○ 하나는 **재실행 단위**임 – 태스크는 실패했을 때 다시
돌리는 최소 단위이므로, 정제와 집계를 한 태스크로 묶으면 집계에서 죽을 때마다 정제 비용을 다시 냄 ○
다른 하나는 **실패 격리**임 – 입력 검증은 파일이 없어서 실패하고 품질 점검은 데이터가 규칙을 어겨서
실패하는데, 둘을 묶으면 실패 상태만 보고는 원인을 구분할 수 없음 – `check_quality` 를 집계에서 떼어 낸
이유가 이것임

□ `check_quality` 가 하는 일은 4절의 총계 보존식 검사임 ○ 등식이 깨지면 이 태스크가 스스로 예외를
던지고, 하류의 `write_report` 는 실행되지 않음 ○ **틀린 보고서를 만든 뒤 회수하는 것이 아니라 애초에
만들지 않는 것이 품질 게이트의 목적**이다

□ 세 번째 코드를 실행함. 같은 그래프를 표준 라이브러리만으로 만든 작은 실행기로 돌림

```
python3 code/6-4-dag-order.py
```

출처: practice/chapter6/results/6-4-dag-order.log

[1] 의존성 선언에서 나오는 실행 순서

validate_input → clean_and_map → aggregate_daily → check_quality → write_report

순서가 정해진 쌍 9개 / 정해지지 않은 쌍 1개

순서 없음: aggregate_daily 와 check_quality - 실행마다 선후가 달라질 수 있음

[2] 2026-07-01 - 입력이 있는 날

dagrun=success

validate_input success

clean_and_map success

aggregate_daily success

check_quality success

write_report success

남은 산출물 6개: validated.json, cleaned.jsonl, quality.json, daily_summary.json, quality_check.json, report.md

[3] 2026-07-04 - 입력 파일이 없는 날

dagrun=failed

validate_input failed FileNotFoundError: 입력 파일 없음: 2026-07-04.jsonl

clean_and_map upstream_failed validate_input 이(가) 끝나지 않음

aggregate_daily upstream_failed clean_and_map 이(가) 끝나지 않음

check_quality upstream_failed clean_and_map 이(가) 끝나지 않음

write_report upstream_failed aggregate_daily 이(가) 끝나지 않음

남은 산출물 0개: 없음

[4] 정제에 누락 결함을 심은 2026-07-01 - 품질 게이트가 막는가

주입한 결함: 매핑 실패 건수를 세지 않고 버린다(drop_silently=True)

dagrun=failed

validate_input success

clean_and_map success

aggregate_daily success

check_quality failed ValueError: 총계 보존 실패: $20 \neq 18 + 1 + 0 + 0$

write_report upstream_failed check_quality 이(가) 끝나지 않음

남은 산출물 4개: validated.json, cleaned.jsonl, quality.json, daily_summary.json

□ 화면에 나온 값에서 읽을 것 ○ 순서가 정해지지 않은 쌍이 하나 있음 — aggregate_daily 와 check_quality 는 둘 다 정제 결과만 있으면 되고 서로를 필요로 하지 않으므로 의존성을 선언하지 않았음. 그래서 이 둘의 선후는 실행마다 달라질 수 있고, 이것은 오류가 아님. 선언하지 않은 순서는 엔진이 보장하지 않는다 ○ 입력이 없는 날 실행된 태스크는 하나뿐임 — 나머지 넷은 실행됐다가 실패한 것이

아니라 `upstream_failed` 로 아예 실행되지 않았음. 예측을 "다섯 개 모두 failed"로 적었다면 여기서 갈림 ○ **산출물이 0개임** — 하류가 돌지 않았으므로 절반쯤 쓰다 만 보고서가 남지 않음. 그런 파일이 남으면 다음 주간 롤업에 섞여 들어감 ○ **누락 결함을 심으면 게이트에서 멈춤** — 매핑 실패 1건을 세지 않게 만들자 20 $\neq 18 + 1 + 0 + 0$ 으로 등식이 깨지고 `check_quality` 가 예외를 던졌음. 집계 파일까지는 만들어졌지만 `report.md` 는 없음 ○ **실패가 상태로 남음** — 4주차의 조용한 유실이나 5주차의 조용한 폐기와 달리 이 실패는 시끄러움. 상태 기록이 없으면 "돌지 않았다"와 "돌 필요가 없었다"를 구분할 수 없음

□ 같은 그래프를 실제 Airflow 3.3.0 엔진으로 돌린 과거 실행도 같은 상태를 남겼음

출처: `practice/chapter6/data/output/ch6_batch_run_report.json` (과거 실행 산출물, start 필드 생략)

```
"failure_run": {
  "dagrun_state": "failed",
  "logical_date": "2026-07-04",
  "tasks": [
    {"task_id": "validate_input", "state": "failed"},
    {"task_id": "clean_and_map", "state": "upstream_failed"},
    {"task_id": "aggregate_daily", "state": "upstream_failed"},
    {"task_id": "check_quality", "state": "upstream_failed"},
    {"task_id": "write_report", "state": "upstream_failed"}
  ]
}
```

- 입력 파일 없음은 재시도로 낫지 않는 실패다
- 원천이 몇 분 늦는 상황이라면 재시도가 파일 도착을 기다리는 장치가 된다.
- 파일이 아예 오지 않는 날이라면 재시도는 실패 통보를 늦추기만 한다. 어느 쪽인지는 원천의 지연 패턴이 정한다.

□ 마지막으로 **logical date**는 그 실행이 처리하는 데이터 구간의 날짜임 ○ 7월 1일 데이터를 처리한 실행의 logical date는 2026-07-01이지만 실제로 돌아간 날은 다를 수 있음 ○ 2주차의 이벤트 시간과 처리 시간 구분이 배치로 오면 "데이터의 날짜"와 "실행된 시각"의 구분이 됨 ○ 이 이름표가 있어야 "7월 1일 실행을 다시 돌려라"가 어느 입력을 읽고 어느 산출물을 덮어쓸지 확정된 명령이 됨

6. 재현성과 롤업 — 다시 계산해도 같은 바이트

□ 배치를 다시 돌릴 수 있다는 말은 다시 돌린 결과가 원래 것과 같을 때만 의미가 있음 ○ 감사에서 "그 통계를 다시 만들어 보라"는 요구를 받았는데 값이 달라지면 어느 쪽이 진본인지를 두고 다투게 됨 ○ 그래서 산출물의 해시(hash)를 기록해 두고 같은 날짜를 다시 돌린 뒤 비교함

□ 같은 바이트는 저절로 나오지 않고 세 가지 설계에서 나옴 ○ 보고서에 생성 시각 같은 비결정 요소를 넣지 않아 내용이 입력만의 함수가 되게 함 ○ JSON 직렬화의 키 순서를 고정해 같은 내용이 다른 바이트가 되는 일을 막음 ○ 산출물을 이어 붙이지 않고 덮어씀. 덮어쓰기는 4주차 UPSERT의 파일판임

□ 주간 롤업은 여기서 한 층 위로 올라감 ○ **각 계층은 아래 계층의 확정 산출물만 읽음** ○ 주간 집계가 원천을 다시 정제하면 비용을 두 번 내고, 정제 규칙이 어긋나는 순간 일별 합과 주간 값이 달라짐 ○ 어느 날짜가 확정되었는지는 `_SUCCESS` 마커(marker)로 판별함 – 일별 배치의 첫 태스크가 시작할 때 마커를 지우고, 마지막 태스크가 성공했을 때만 다시 씀 – 마커가 있다는 것은 그날의 배치가 품질 게이트까지 통과해 끝났다는 뜻임

□ 네 번째 코드를 실행함

```
python3 code/6-5-rerun-rollup.py
```

출처: practice/chapter6/results/6-5-rerun-rollup.log

[1] 2026-07-01 보고서를 두 번 만들어 파일 해시를 비교 - 입력만의 함수일 때

1회차 3e2de6b0dd9b5813...

2회차 3e2de6b0dd9b5813...

같은 바이트인가: 예

[2] 같은 보고서에 생성 시각 한 줄을 넣으면

달라진 줄: '- 생성 시각: ...' 한 줄 (실행할 때마다 값이 바뀜)

같은 바이트인가: 아니오

→ 집계 결과가 같아도 해시 비교로는 '재현됐다'를 보일 수 없게 됨

[3] 주간 창 2026-06-29 ~ 2026-07-05 - _SUCCESS 마커로 확정 여부를 판별

2026-06-29 마커 없음 일별 요약 파일 없음

2026-07-01 마커 있음 일별 요약 파일 있음

2026-07-02 마커 있음 일별 요약 파일 있음

2026-07-03 마커 있음 일별 요약 파일 있음

2026-07-04 마커 없음 일별 요약 파일 없음

합산 대상 3일, 결측 4일: 2026-06-29, 2026-06-30, 2026-07-04, 2026-07-05

[4] 일별 확정 산출물만 합산한 주간 값 - 원천을 다시 읽지 않음

총계 보존 $60 = 54 + 2 + 3 + 1$ OK

지역별 마포구 16, 관악구 15, 강남구 23

유형별 도로 21, 소음 15, 청소 8, 환경 10

검산 지역별 합 54 / 유형별 합 54 / 매핑 54

[5] 주간 쪽이 원천을 다시 정제하는데 정제 규칙만 구버전이면

구버전 사전: 약칭 ['강남', '마포'] - '관악' 항목이 아직 없음

원천 재정제 결과 매핑 52건 / 미매핑 5건

일별 합산 결과 매핑 54건 / 미매핑 3건

같은 주에 대해 두 보고서가 매핑 2건 차이

두 계산 모두 총계 보존은 성립함 - 각자 안에서는 합이 맞아 오류로 잡히지 않음

□ 화면에 나온 값에서 읽을 것 ○ 같은 입력으로 두 번 만든 보고서의 해시가 같음 — 3e2de6b0... 이 두 번 나왔음. "다시 해 봤더니 비슷했다"가 아니라 바이트가 같음을 한 줄로 보일 수 있음 ○ 생성 시각 한 줄이 그 성질을 깨뜨림 — 집계 결과가 완전히 같은데도 해시가 달라짐. 재현성은 계산이 맞는가와 별개로 비교 가능한 형태로 쓰였는가의 문제임 ○ 마커가 있는 날만 합산됨 — 주간 창 7일 중 3일만 합산되고 나머지 4일은 결측으로 적혔음. 결측을 적지 않고 그냥 3일치를 합하면 "이번 주 민원 60건"이 7일치처럼 읽힘 ○ 지역별 합과 유형별 합이 모두 54로 매핑 건수와 맞음 — 롤업 단계에서도 검산이 성립함. 이 검산이 없으면

상위 집계에서 새로 생긴 누락을 발견할 수 없음 ○ 규칙이 한쪽만 구버전이면 2건이 갈림 — 주간 쪽 약칭 사전에서 관악 한 항목이 빠졌을 뿐인데 매핑이 54건과 52건으로 나뉨. 두 보고서 모두 자기 안에서는 총계 보존이 성립하므로 어느 쪽도 오류로 잡히지 않음

□ 과거 Airflow 실행도 같은 성질을 기록으로 남겼음 ○ 7월 1일을 다시 돌린 뒤 `daily_summary.json` 과 `report.md` 의 해시가 재실행 전후로 같았고, `identical` 이 `true` 로 적혔음

(`practice/chapter6/data/output/ch6_batch_run_report.json` 의 `idempotency_check`)

- 재현성은 한 번 확인하고 끝나는 성질이 아니다
- 정제 규칙이나 보고서 서식을 고치면 그날부터 같은 입력이 다른 바이트를 낸다.
- 의도한 변경이면 "확정치의 계산 규칙이 바뀐 날짜"로 기록하고, 의도하지 않은 것이면 사고다. 해시 비교는 둘을 가르는 장치다.

7. 핵심 요약

□ 1. 배치의 마감은 달력이 정하고, 그래서 구간 전량을 한 번에 본다. (3절) ○ 세 파일 모두 다른 날짜의 레코드가 섞이지 않았고, "7월 1일분"이 무엇인지를 파일 이름이 정했음 ○ 7월 2일의 중복 두 건은 간격이 192분이어서 5주차의 10분 창으로는 만나지 못하지만, 하루치를 통째로 켜 배치는 지을 수 있음 □ 2. 정제한 레코드는 전부 행방을 적고, 합이 맞는지 식으로 검사한다. (4절) ○ 사흘치가 $60 = 54 + 2 + 3 + 1$ 로 남김없이 설명됐고, 일별로도 20:18:22 모두 등식이 성립했음 ○ 매핑 실패를 세지 않는 코드는 총계 54건만 적고 종료 코드 0으로 끝나, 차이 6건이 로그에도 산출물에도 남지 않음 □ 3. 키를 이름에서 표준 코드로 바꾸면 집계가 모이고, 미기재와 미매핑은 합치지 않는다. (4절) ○ 이름을 키로 쓰면 13줄이고 강남 계열만 4줄로 쪼개지지만, 표준 코드를 키로 쓰면 3줄이 됨 ○ 미기재의 조치 대상은 접수 시스템이고 미매핑의 조치 대상은 사전이므로, 판교 · 성남시 · 김포 를 비슷한 자치구로 끼워 맞추는 것은 데이터 조작이다 □ 4. 품질 게이트는 틀린 산출물을 회수하지 않고 애초에 만들지 않는다. (5절) ○ 입력이 없는 날 실행된 태스크는 `validate_input` 하나뿐이었고 나머지 넷은 `upstream_failed` 로 남아 산출물이 0개였음 ○ 누락 결함을 십자 $20 \neq 18 + 1 + 0 + 0$ 에서 멈춰 집계 파일까지만 남고 `report.md` 는 만들어지지 않았음 □ 5. 같은 입력의 재실행은 같은 바이트를 내야 하고, 상위 집계는 확정 산출물만 읽는다. (6절) ○ 생성 시각을 넣지 않은 보고서는 두 번 만들어도 해시가 `3e2de6b0...` 으로 같았고, 시각 한 줄을 넣자 달라졌음 ○ 주간 롤업은 `_SUCCESS` 마커가 있는 3일만 합산했고, 규칙이 한쪽만 구버전이면 같은 주의 매핑이 54건과 52건으로 갈렸음

8. 과제

□ 실습을 순서대로 실행해 만든 산출물 네 개를 LMS에 제출한다

1. 표 6.2 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.
2. `practice/chapter6/results/6-3-clean-accounting.log`

3. `practice/chapter6/results/6-4-dag-order.log`
4. `practice/chapter6/results/6-5-rerun-rollup.log`

□ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 6-3 로그의 [4] 에서 **날짜 하나**를 골라 `physical · dup · valid · mapped · missing · unmapped` 를 옮겨 적고, 총계 보존식이 성립함을 산술로 보인다.
2. `missing` 과 `unmapped` 를 합쳐 "실패 2건"으로 보고하면 안 되는 이유를, 두 항의 조치 대상과 연결해 쓴다.
3. 6-4 로그의 [3] 에서 실행된 태스크와 실행되지 않은 태스크를 나눠 적고, 남은 산출물이 0개인 것이 왜 중요한지 쓴다.
4. 6-5 로그의 [5] 에서 두 매핑 건수를 옮겨 적고, 두 보고서 모두 총계 보존이 성립하는데도 문제인 이유를 쓴다.

□ 값을 지어내지 않는다 □ 실행이 안 되면 어디서 막혔는지 그대로 쓴다 □ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

□ 실습을 더 해 보려면 `practice/chapter6/` 에서 다음을 시도함 ○ 6-3-clean-accounting.py 의 ALIASES 에 표기를 하나 더 넣고 미매핑 3건이 어떻게 바뀌는지 확인함 ○ 6-4-dag-order.py 의 DEPENDS_ON 에서 `check_quality` 를 `write_report` 의 상류에서 빼고, 결함을 심은 실행이 보고서를 만들어 버리는지 확인함 ○ 6-5-rerun-rollup.py 의 WINDOW_END 를 바꿔 합산 대상 일수와 결측 일수가 어떻게 달라지는지 봄 ○ macOS·Linux·WSL2를 쓸 수 있으면 `python scripts/setup_practice.py 6` 뒤에 `python run_chapter6.py` 로 같은 절차를 실제 Airflow 엔진으로 돌려 봄 □ 뒤 주차에서 이어지는 내용 ○ 배치가 만든 확정 피처를 어디에 두고 어떻게 다시 쓰는지는 7주차에서 다룸 ○ 재실행과 재현성을 실험·모델 단위로 확장하는 것은 9주차, 산출물 품질을 운영 중에 감시하는 것은 11주차에서 다룸 □ 원문을 직접 보고 싶다면 다음 두 편이 출발점임 ○ Apache Airflow, 「DAGs」 — 태스크 의존성과 실행 상태의 공식 설명. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html> ○ 행정표준코드관리시스템 — 법정동코드를 비롯한 공공 표준 코드의 원천. <https://www.code.go.kr/>